



Day 2: Git & AI-Powered Development Workflow

CI/CD with Jenkins and GenAI Training — Building on Day 1 AWS Foundations

Today's Agenda

01

Git Fundamentals

What Git is, why it matters, and how it works under the hood

02

Branching & Merging

Feature branches, merge strategies, and conflict resolution

03

GitHub Integration

Remote repos, push/pull, pull requests, and team collaboration

04

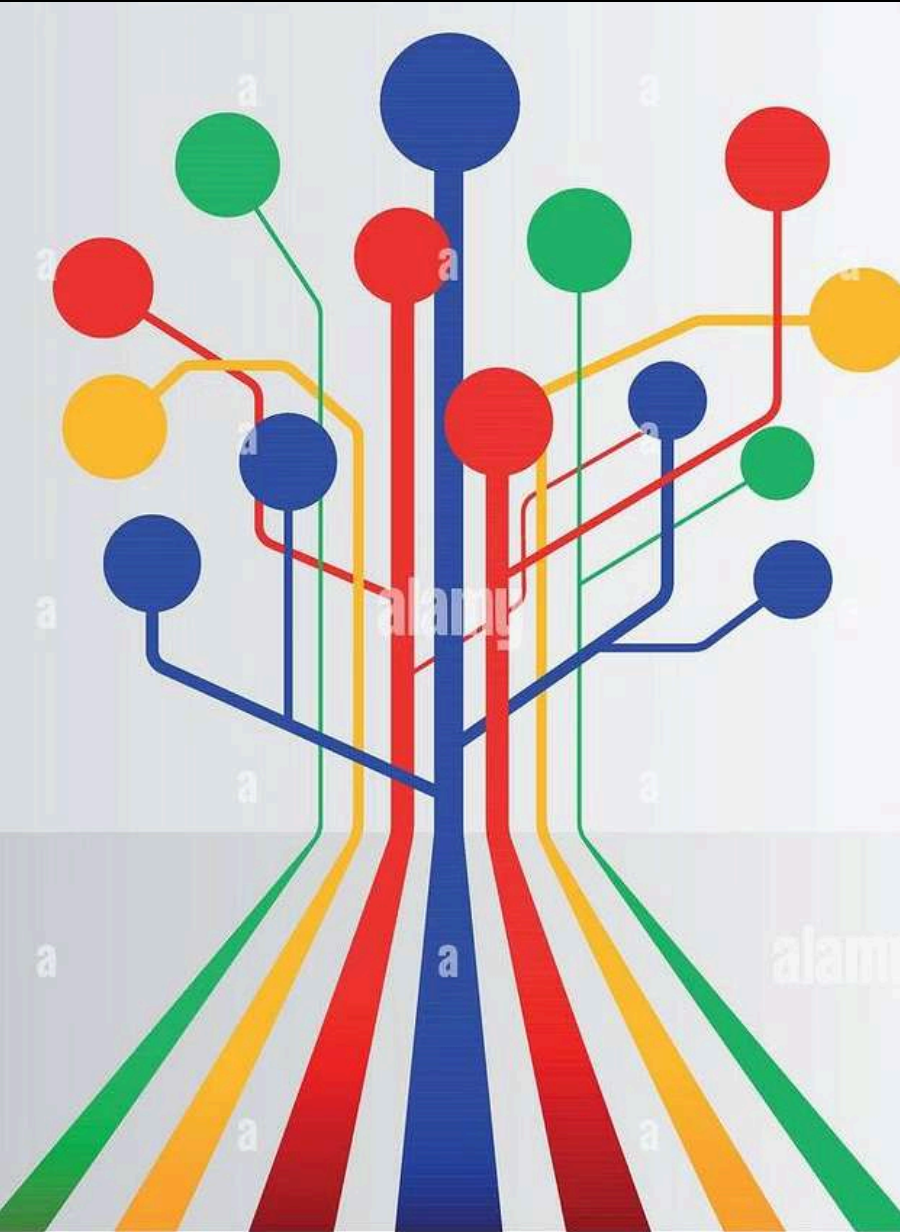
AI in Development

GitHub Copilot, ChatGPT, and AI-assisted coding workflows

05

Mini Project

Build, commit, and push a small app using AI assistance



CHAPTER 1

Git Fundamentals

Version control, distributed workflows, and the foundation of modern software development

What Is Git?



Git is a **distributed version control system** that tracks changes in your codebase over time. Every developer has a full copy of the project history locally.

Track Changes

Every edit is recorded with who, what, and when

Collaborate

Multiple developers work without overwriting each other

Revert Safely

Roll back to any previous state of the project

Why Git? The Problem It Solves

Without Git

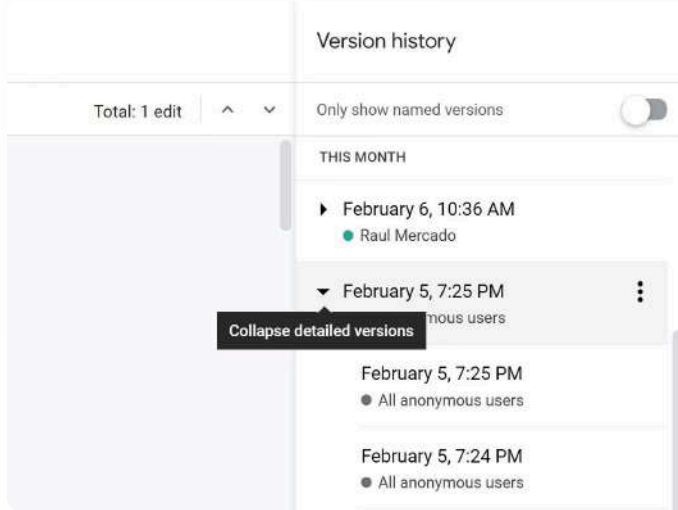
- app_v1_final.zip
- app_v2_REALLY_final.zip
- app_backup_johns_copy.zip
- Lost work, no history, chaos

With Git

- Structured commit history
- Parallel feature branches
- Conflict detection built-in
- Full audit trail, always

 Git is not optional in professional software teams — it is the baseline expectation for every developer and DevOps engineer.

Real-World Analogy: Google Docs Versioning



Think of Git like **Google Docs version history** — but for code, with superpowers.

→ **Every save = a commit**

Each commit is a snapshot of your entire project at that moment

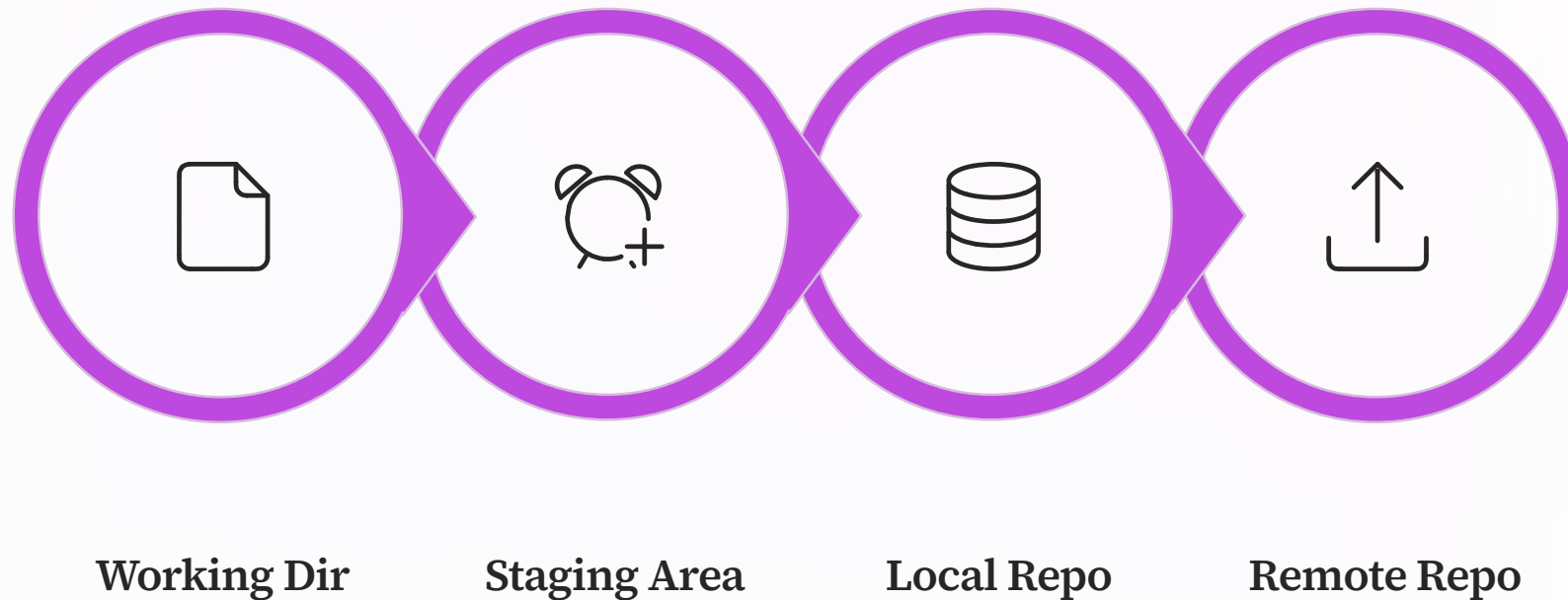
→ **See who changed what**

git blame shows exactly who edited each line and when

→ **Restore any version**

Roll back to any commit — even months-old code

Git Workflow Overview



Understanding these four zones is the key to mastering Git. Every Git command moves code between these stages in a deliberate, controlled way.

CHAPTER 2

Core Git Commands

The essential commands every developer must know by heart



Starting a Repository

Initialize or Clone

```
# Start a new repo from scratch
git init my-project
cd my-project

# OR clone an existing one
git clone https://github.com/user/repo.git
cd repo
```

 **git init** creates a hidden `.git` folder that tracks all history.
git clone copies a full remote repo locally.

git init

Creates a brand-new empty repository in the current directory

git clone

Downloads a full copy of a remote repository including all history

Staging & Committing Changes

Check what has changed
git status

Stage **specific** file
git **add** app.js

Stage **ALL** changed files
git **add** .

Commit with a meaningful message
git **commit** -m "feat: add user login endpoint"

git status

Shows which files are modified, staged, or untracked

git add

Moves changes to the staging area — ready to commit

git commit

Saves staged snapshot permanently to local history

Working with Remote Repositories

```
# Link local repo to GitHub remote  
git remote add origin https://github.com/user/repo.git
```

```
# Send local commits to remote  
git push origin main
```

```
# Get latest changes from remote  
git pull origin main
```

git push

Uploads your local commits to the remote — shares work with the team

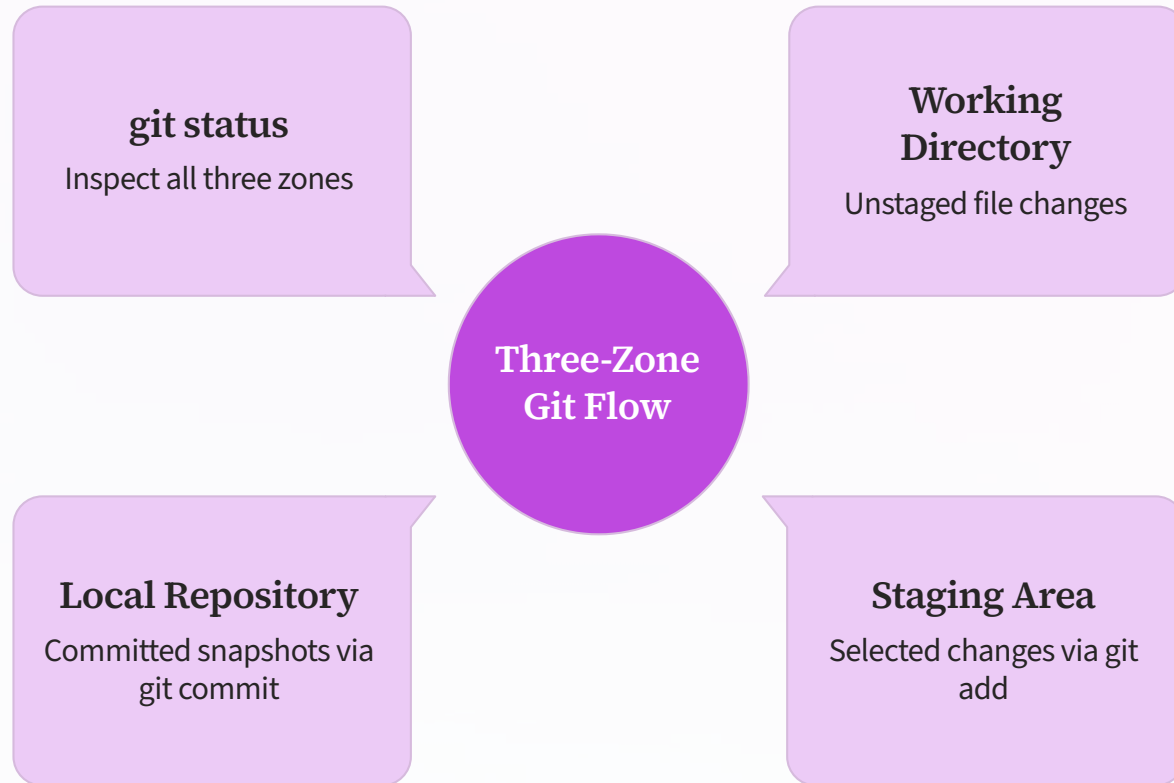
git pull

Downloads remote changes and merges them into your local branch



Always pull before you push. Fetching remote updates first prevents unnecessary merge conflicts with your teammates' work.

Working Directory → Staging → Repo



The staging area is Git's most powerful feature — it lets you carefully choose exactly which changes to group into each commit, creating a clean, readable history.

Essential Commands at a Glance



Inspect

`git log --oneline` — compact commit history

`git diff` — see unstaged changes



Undo

`git restore file.js` — discard working changes

`git reset HEAD~1` — undo last commit



Tag

`git tag v1.0.0` — mark a release version

`git push --tags` — push tags to remote



Configure

`git config --global user.name`

`git config --global user.email`

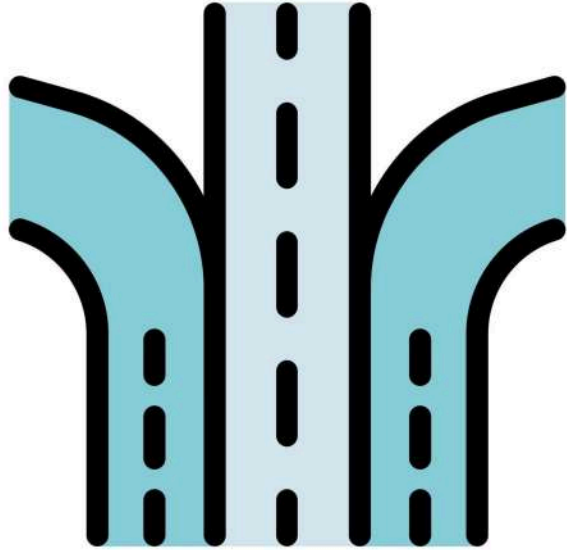


CHAPTER 3

Branching & Merging

Parallel development, feature isolation, and safe integration strategies

What Is a Branch?



A branch is an **independent line of development**. It lets you work on features, fixes, or experiments without touching the stable codebase.

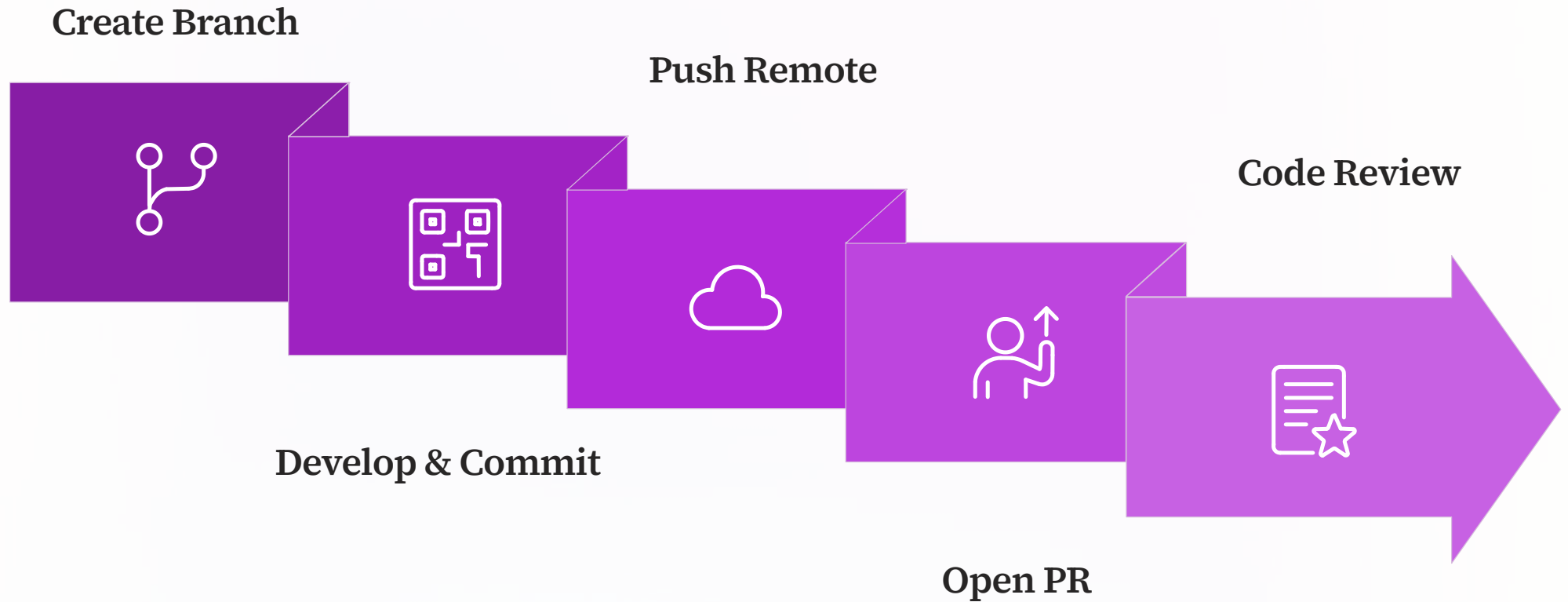
```
# Create and switch to new branch  
git checkout -b feature/login-page
```

```
# List all branches  
git branch -a
```

```
# Switch between branches  
git checkout main
```

✓ Branches are lightweight and fast in Git — creating one takes milliseconds.

Feature Branch Workflow



This workflow is the industry standard for team collaboration. Every new feature or bug fix gets its own branch, keeping `main` always stable and deployable.

Merging Branches

```
# Switch to the target branch
```

```
git checkout main
```

```
# Merge feature branch into main
```

```
git merge feature/login-page
```

```
# Delete branch after merge (clean up)
```

```
git branch -d feature/login-page
```

Fast-Forward Merge

No divergent history — Git simply moves the pointer forward. Clean and simple.

3-Way Merge

Both branches have unique commits — Git creates a merge commit to combine them.

Squash Merge

Combines all feature commits into one clean commit on main. Preferred by many teams.

Merge Conflicts: What & Why

When Do Conflicts Occur?

A conflict happens when two branches modify the **same line** of the same file differently. Git cannot auto-decide which version to keep.

```
<<<<<<< HEAD
  console.log("Hello from main");
=====
  console.log("Hello from feature");
>>>>>> feature/login-page
```

Step 1: Identify

Run `git status` to list all conflicted files

Step 2: Resolve

Open the file, choose the correct version, remove conflict markers

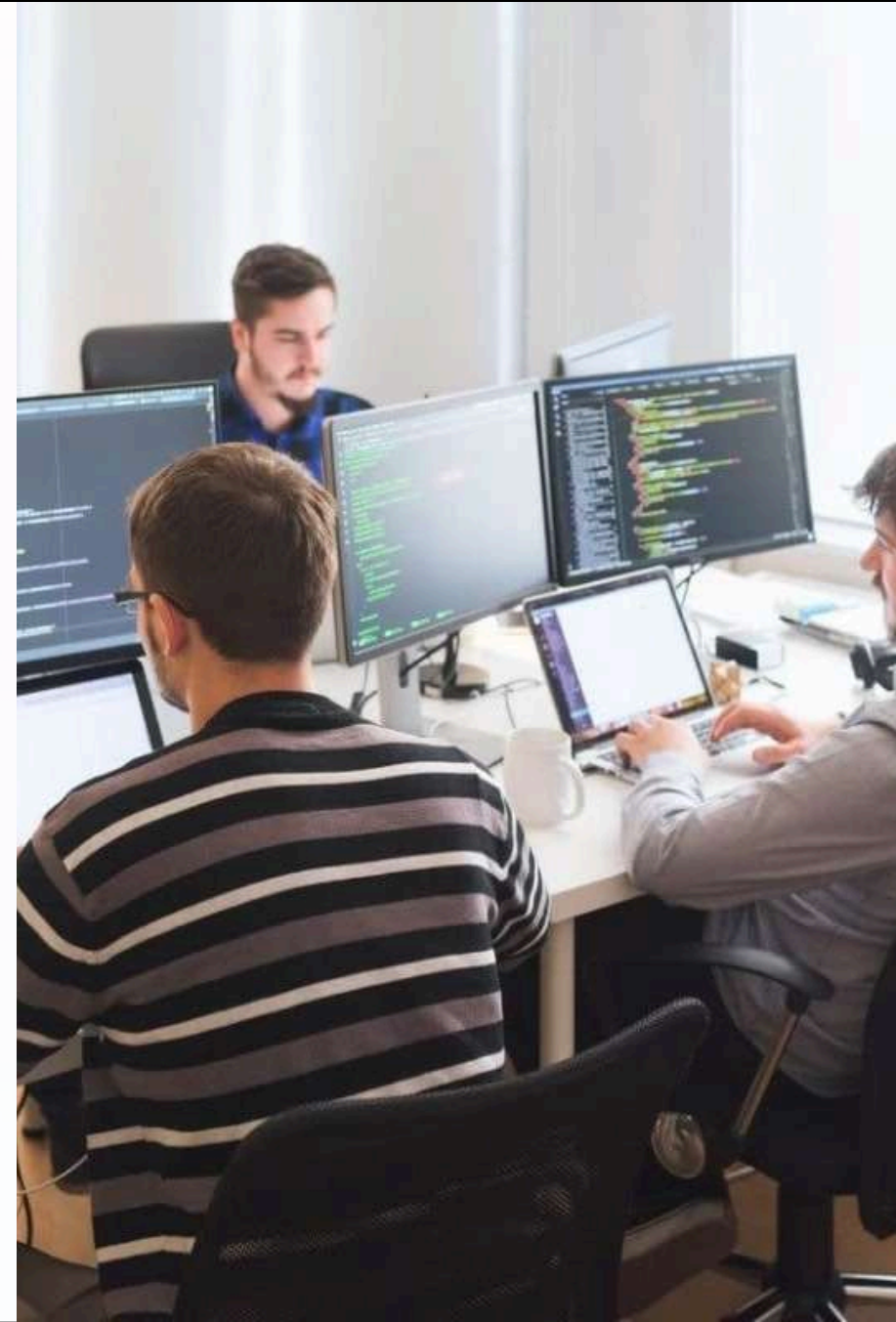
Step 3: Commit

Run `git add .` then `git commit` to finalize

Real-World Team Scenario

Scenario: Alice builds the login feature on `feature/login`. Bob fixes a bug on `hotfix/null-check`. Both work simultaneously — neither blocks the other. When ready, both open pull requests to `main`, get reviewed, and merge independently.

- ✓ This parallel workflow is how teams of 2 or 2,000 ship code without stepping on each other.



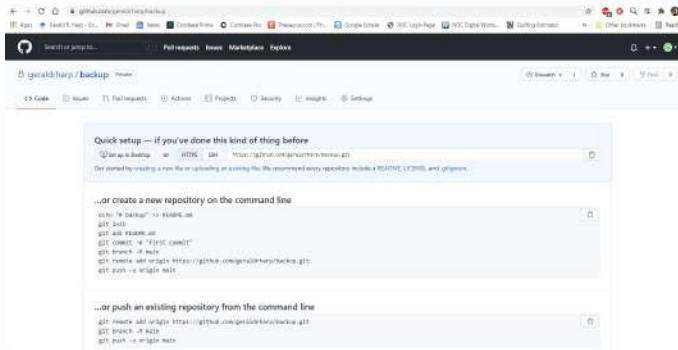
GitHub

CHAPTER 4

GitHub Integration

Remote repositories, collaboration, and connecting to your CI/CD pipeline

What Is GitHub?



GitHub is a **cloud-hosted platform** for Git repositories. It adds collaboration features on top of Git's version control foundation.



Remote Hosting

Your code is backed up and accessible anywhere



Team Collaboration

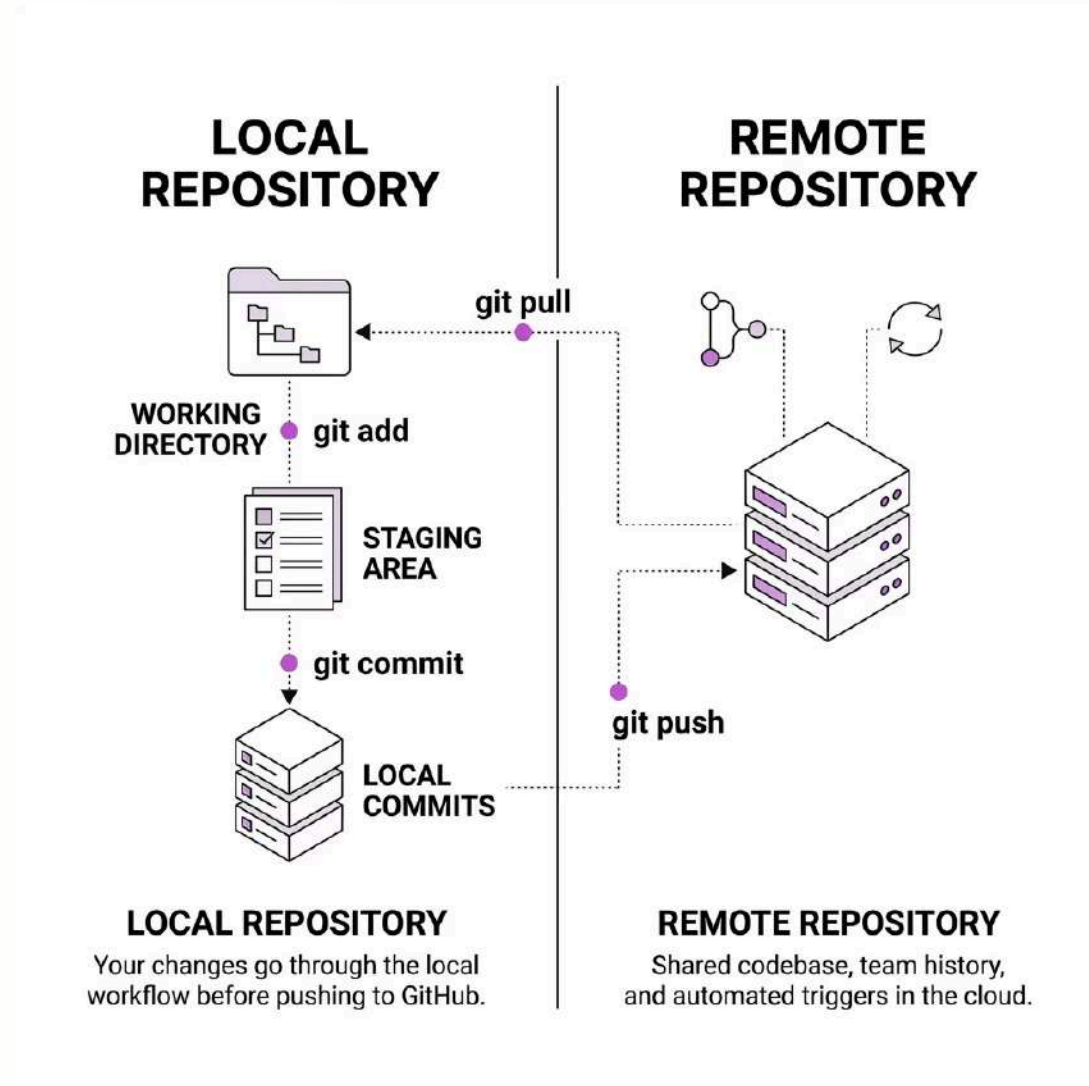
Pull requests, code reviews, and issue tracking



CI/CD Integration

Webhooks trigger Jenkins builds on every push

Local vs Remote Repository



Your **local repo** is your private workspace. The **remote repo** on GitHub is the source of truth shared by the entire team.



Think of local as your draft notebook and GitHub as the published, shared version of the project.

Push Code to GitHub

1. Create repo on GitHub first, then link it

```
git remote add origin https://github.com/yourname/my-app.git
```

2. Rename default branch to main

```
git branch -M main
```

3. Push and set upstream tracking

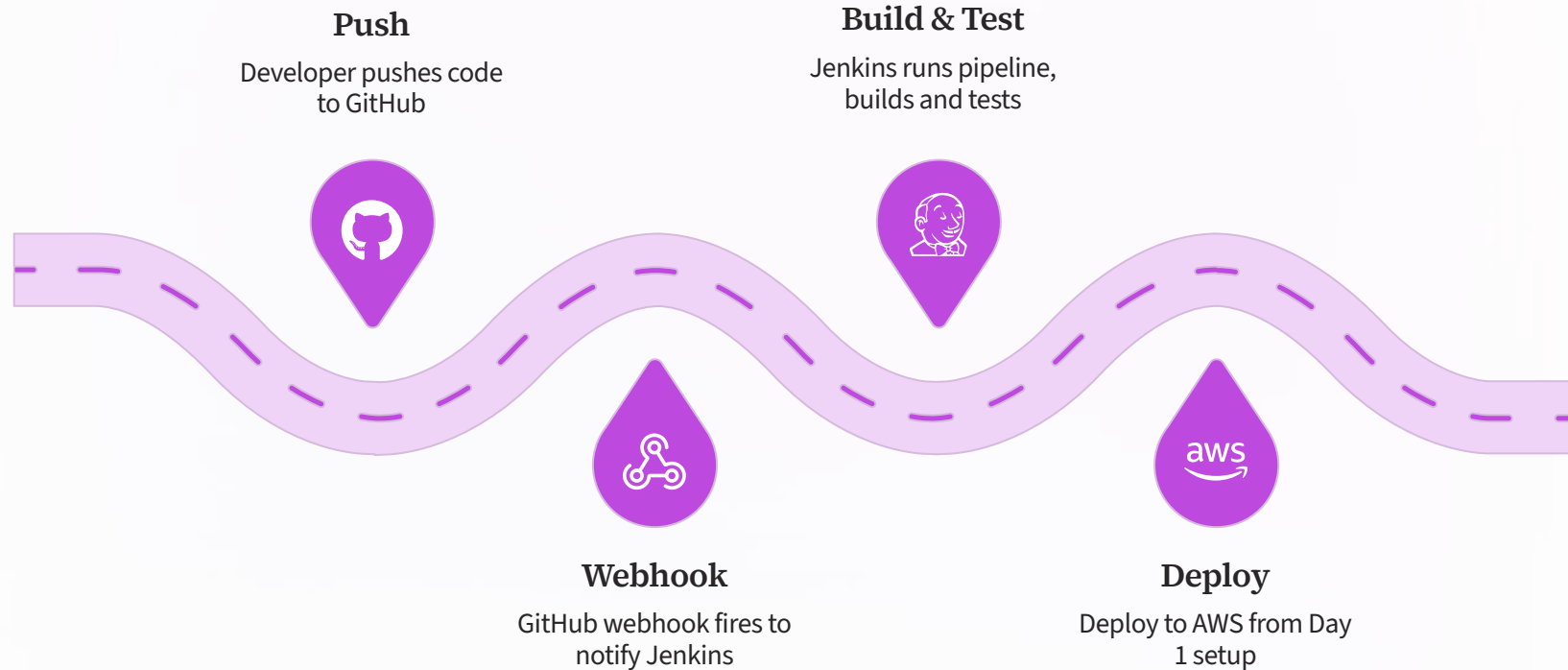
```
git push -u origin main
```

4. Future pushes are now short

```
git push
```

✔ The `-u` flag sets the upstream so future `git push` commands require no extra arguments.

GitHub → Jenkins: CI/CD Preview



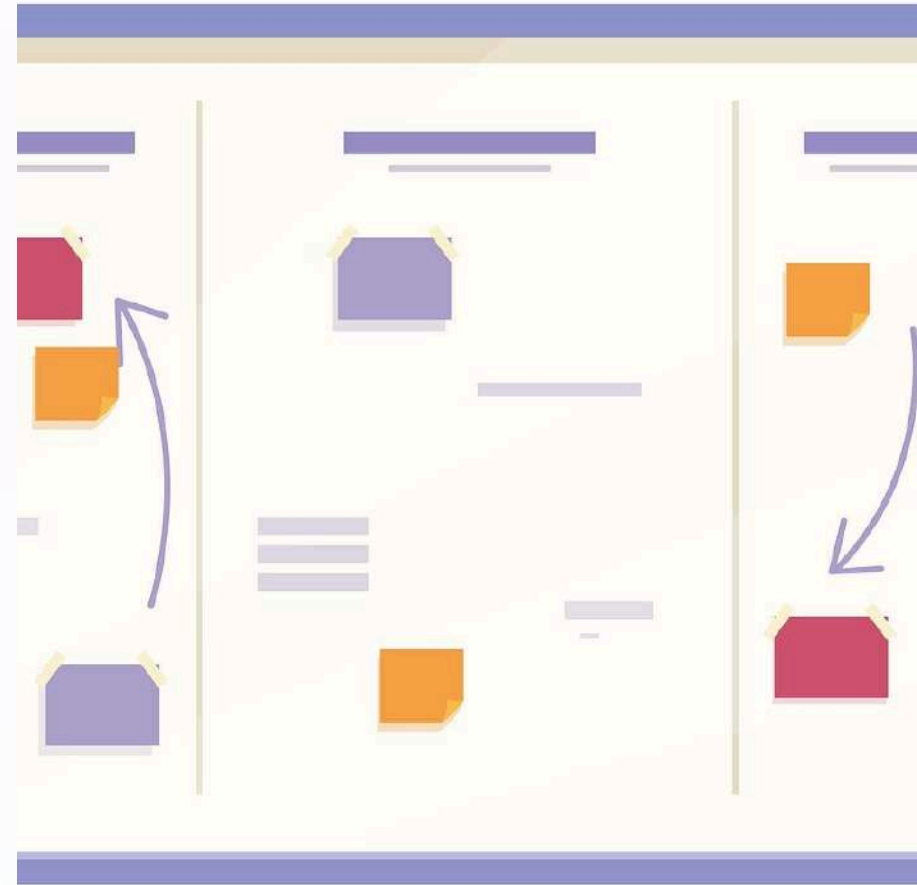
This is the automation bridge you'll build on Day 3. Every git push becomes an automatic trigger — your pipeline runs without any manual intervention.

- ❏ Day 3 will cover Jenkins pipeline configuration and connecting it to this GitHub repository.

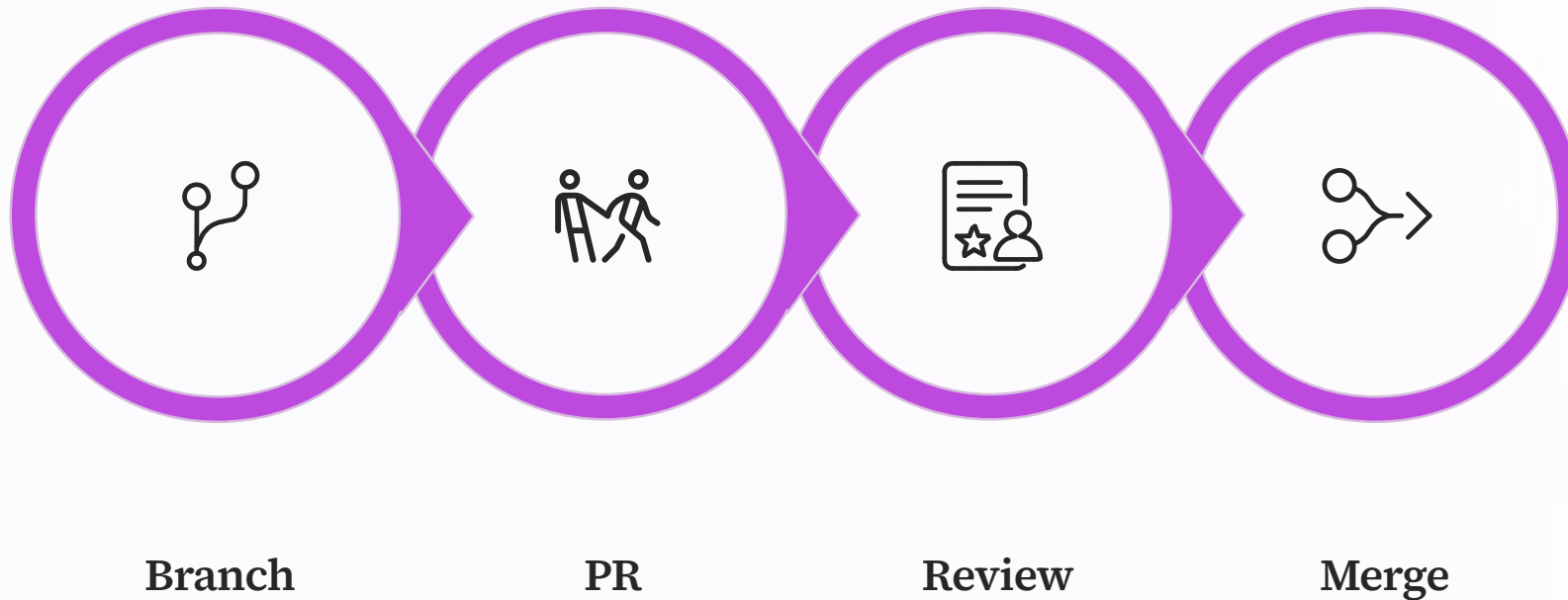
CHAPTER 5

Git Workflow in Teams

Pull requests, code reviews, and how DevOps teams collaborate at scale

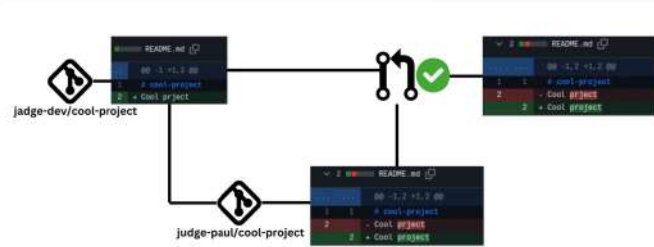


The Complete Team Workflow



This is the **Pull Request (PR) workflow** — the gold standard for professional teams. No code reaches `main` without at least one review, ensuring quality and knowledge sharing.

The Role of Pull Requests



Basic Git Workflow

What Happens in a PR?

→ Diff View

Reviewers see exactly which lines changed — additions in green, deletions in red

→ Inline Comments

Feedback is tied to specific lines of code — precise and actionable

→ CI Status Checks

Automated tests must pass before merge is allowed — safety net for the team

Team Collaboration Model

Developer

Creates branch, writes code, commits, and opens a PR for review

Reviewer

Reads the diff, leaves comments, approves or requests changes

CI Bot

Runs tests, checks linting, and reports pass/fail directly on the PR

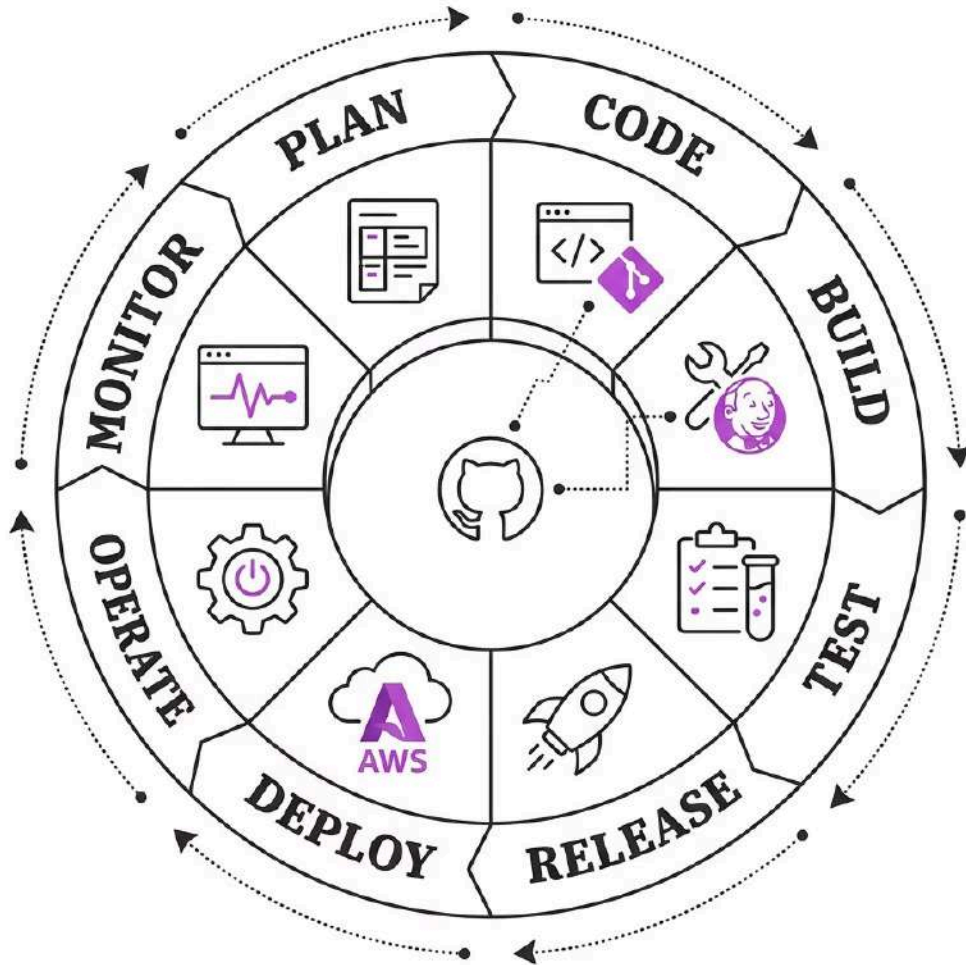
DevOps

Merges approved PRs — triggers deployment pipeline automatically



Even a team of one benefits from this model — PRs create an automatic log of every decision made during development.

DevOps Workflow Connection



Git and GitHub are the **entry point** of the entire DevOps loop. Every automated step downstream — builds, tests, deployments — is triggered by a simple git push.

- You already own the Deploy and Operate steps from Day 1. Today you're completing the Code and Build connection.

CHAPTER 6

AI in Development

GitHub Copilot, ChatGPT, and how AI amplifies your engineering workflow



Why AI Tools Matter for Developers



AI coding assistants don't replace developers — they **eliminate toil** and accelerate the parts of coding that are repetitive or time-consuming.

Faster Boilerplate

Generate setup code in seconds instead of minutes

Learn While Coding

AI explains unfamiliar code or libraries inline

Fewer Context Switches

Stay in the editor — no more hunting through docs

GitHub Copilot Overview

What It Is

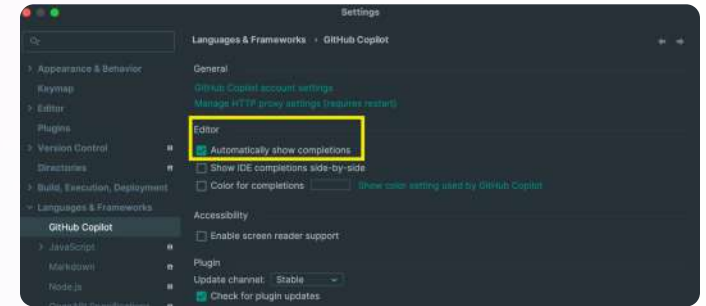
AI pair programmer built into VS Code, JetBrains, and Neovim. Powered by OpenAI Codex.

How It Works

Reads your file context and comments, then auto-suggests complete lines or functions

Key Shortcut

Press `Tab` to accept a suggestion. `Alt+]` to cycle through alternatives.



ChatGPT for Developers



Code Generation

"Write a Python Flask API with GET and POST endpoints for a user resource"



Commit Messages

"Write a conventional commit message for these changes: [paste diff]"



Code Explanation

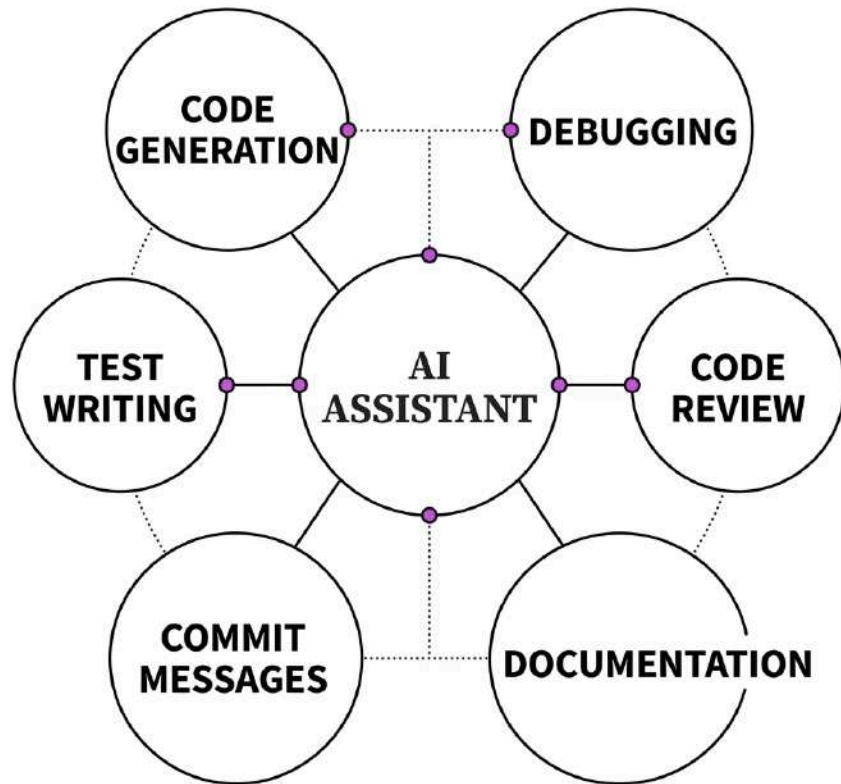
"Explain what this function does and identify any potential bugs"



Documentation

"Generate a README for this Node.js project with setup instructions and API docs"

AI Use Cases: Side by Side



AI assistants are most powerful when used as a **first draft engine**. Always review, test, and understand the output — you own the code, the AI is your co-pilot.



Never paste sensitive credentials, customer data, or proprietary code into public AI tools. Use enterprise or self-hosted options for sensitive workloads.

Crafting Effective AI Prompts

The anatomy of a great developer prompt:

1	2	3	4
Role "Act as a senior Node.js developer..."	Task "...write an Express middleware for JWT authentication..."	Context "...the app uses MongoDB and handles 10k daily users..."	Format "...return only the code with inline comments, no explanations."



The more precise your context, the higher-quality the output. Vague prompts produce generic code.



CHAPTER 7

Live Demo

Watch AI-assisted development in action — then try it yourself

Demo 1: Generate Code with AI

The Prompt

Act as a Node.js developer.
Create an Express.js REST API with:

- GET /health → returns { status: "ok" }
- POST /echo → returns the request body

Include error handling and use ES modules.

 Watch Copilot auto-complete as we type this prompt as a comment in VS Code.

Expected Output

```
import express from 'express';
const app = express();
app.use(express.json());

app.get('/health', (req, res) => {
  res.json({ status: 'ok' });
});

app.post('/echo', (req, res) => {
  res.json(req.body);
});

app.listen(3000);
```

Demo 2: Explain Code with AI

```
// Paste this into ChatGPT:  
// "Explain what this code does, line by line.  
// Identify any security concerns."  
  
const jwt = require('jsonwebtoken');  
module.exports = (req, res, next) => {  
  const token = req.headers['authorization']?.split(' ')[1];  
  if (!token) return res.status(401).json({ error: 'No token' });  
  try {  
    req.user = jwt.verify(token, process.env.JWT_SECRET);  
    next();  
  } catch { res.status(403).json({ error: 'Invalid token' }); }  
};
```

 AI will explain Bearer token extraction, JWT verification, error handling, and flag that `JWT_SECRET` must never be hardcoded.

Demo 3: Generate README with AI

Prompt Template

Generate a professional GitHub README for a Node.js Express REST API project.

Include:

- Project description
- Prerequisites
- Installation steps
- API endpoints table
- How to run tests
- License section (MIT)

Format using Markdown.

Project Title & Badge

CI status badge linked to your GitHub Actions or Jenkins build

Installation

Step-by-step clone, install, and start commands

API Reference

Markdown table of endpoints, methods, and expected responses

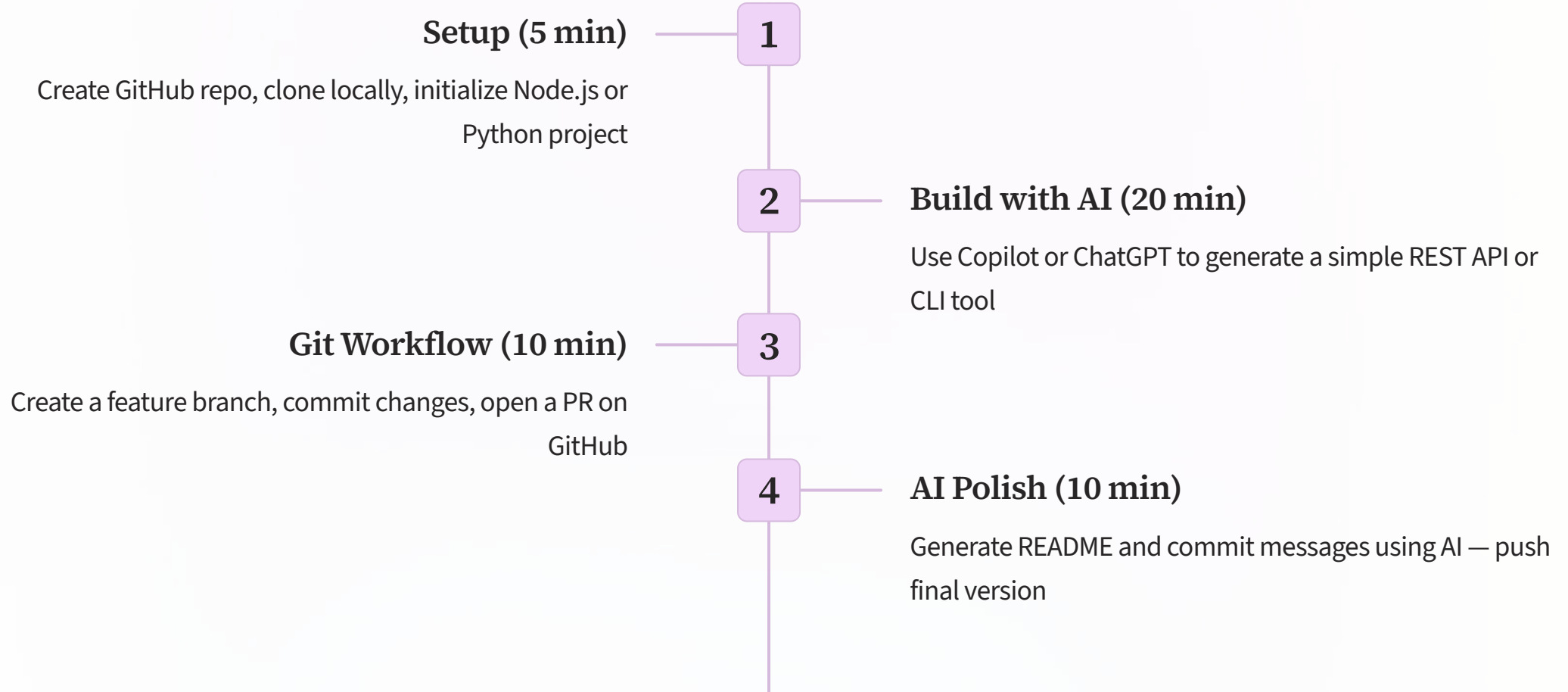
CHAPTER 8

Mini Project

Hands-on time — build, commit, and push a real app with AI assistance



Mini Project Overview



Option A: Node.js Express API

```
# Initialize project
mkdir my-api && cd my-api
git init
npm init -y
npm install express

# Use AI to generate: server.js with /health and /items endpoints
# Commit each logical step separately

git add .
git commit -m "feat: initial Express server setup"
git checkout -b feature/items-endpoint
# ... add /items route ...
git commit -m "feat: add GET /items endpoint"
git push origin feature/items-endpoint
```

 Push the feature branch and open a PR on GitHub — even if you're solo. Practice the workflow!

Option B: Python Flask API

```
# Initialize project
mkdir flask-api && cd flask-api
git init
python3 -m venv venv && source venv/bin/activate
pip install flask

# Use AI to generate: app.py with /health and /tasks endpoints
# Create requirements.txt
pip freeze > requirements.txt

git add .
git commit -m "feat: initial Flask app with health check"
git checkout -b feature/tasks-endpoint
# ... add /tasks route with in-memory list ...
git commit -m "feat: add CRUD endpoints for tasks"
git push origin feature/tasks-endpoint
```

Mini Project: Git Checklist

- ☐ Created a new GitHub repository (public)
- ☐ Cloned the repo locally with git clone
- ☐ Created a feature branch with git checkout -b
- ☐ Made at least 3 meaningful commits with clear messages
- ☐ Used AI to generate at least one code file
- ☐ Pushed the feature branch to GitHub
- ☐ Opened a Pull Request on GitHub
- ☐ Generated a README using AI and committed it
- ☐ Merged the PR into main

Commit Message Best Practices

Use Conventional Commits format:

feat: New feature

feat: add POST /tasks endpoint with validation

fix: Bug fix

fix: handle null body in echo endpoint

docs: Documentation

docs: add README with setup and API reference

chore: Maintenance

chore: add .gitignore for node_modules



AI Tip: Paste your `git diff --staged` output into ChatGPT and ask it to write a conventional commit message for you.



CHAPTER 9

Summary & Outcomes

What You Achieved Today



Git Mastery

init, clone, add, commit, push, pull — the full local and remote workflow



Branching & PRs

Feature branch workflow, merge strategies, and conflict resolution



GitHub Collaboration

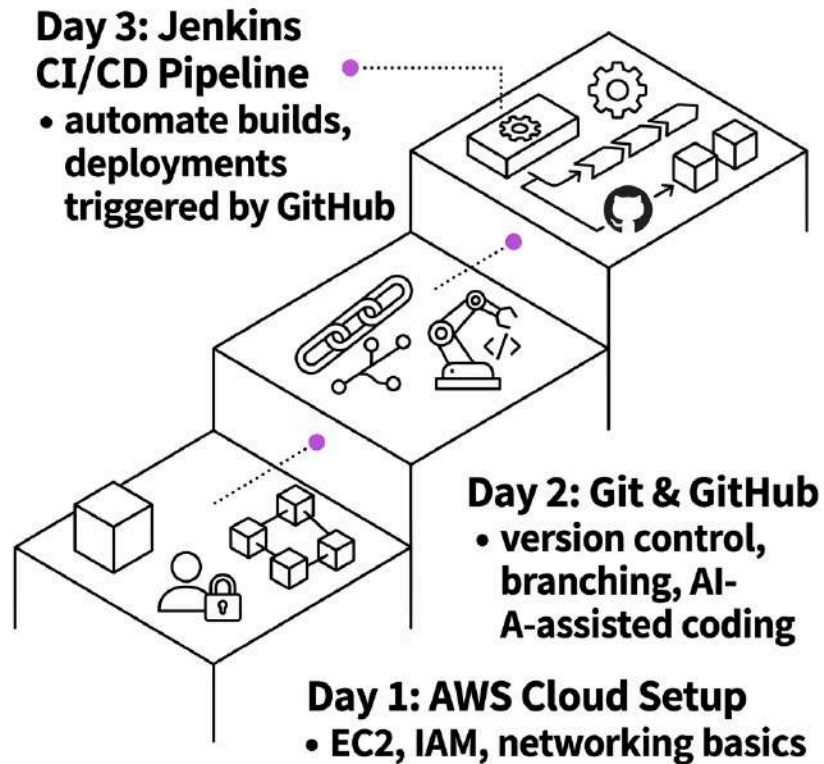
Remote repositories, pull requests, and professional team workflows



AI-Powered Dev

Copilot and ChatGPT for code generation, explanation, and documentation

Your Learning Stack – Day by Day



Each day builds directly on the last. Your GitHub repo from today becomes the **source trigger** for the Jenkins pipeline you'll build tomorrow.

 **Day 3 Preview:** You'll configure a Jenkins job that watches your GitHub repo and automatically builds and deploys on every push to `main`.

Connecting the Dots: Git → Jenkins



The GitHub repository you built today is already **pipeline-ready**. On Day 3, you'll add a Jenkinsfile to the root of your repo and wire up this entire automated flow from scratch.

Day 3 Sneak Peek: Jenkins Pipeline

What You'll Build Tomorrow

```
pipeline {  
  agent any  
  stages {  
    stage('Build') {  
      steps { sh 'npm install' }  
    }  
    stage('Test') {  
      steps { sh 'npm test' }  
    }  
    stage('Deploy') {  
      steps { sh './deploy.sh' }  
    }  
  }  
}
```

Jenkinsfile

Pipeline-as-code stored in your GitHub repo — versioned with your application

Webhook Trigger

GitHub notifies Jenkins on every push — no manual builds ever

AWS Deploy

Jenkins SSHs into your EC2 instance from Day 1 and deploys automatically

Key Takeaways



Git is the foundation

Every professional dev workflow starts here. Master the core commands cold.



Branches protect stability

Never commit directly to main. Branch → build → review → merge.



AI amplifies, not replaces

Use AI to eliminate toil — always review and own every line of generated code.

Before Day 3 — Homework Checklist

- ☐ Your mini project repo is on GitHub with a clean commit history
- ☐ A feature branch was created, pushed, and merged via PR
- ☐ A README exists in the repo (AI-generated is fine)
- ☐ You've practiced `git log --oneline` to verify your history
- ☐ Review the Jenkinsfile syntax (linked in course materials)
- ☐ Ensure your AWS EC2 instance from Day 1 is still running

 Come to Day 3 with your GitHub repo URL ready — it's the first thing you'll need when configuring Jenkins.

Resources & Further Reading

Git Docs

git-scm.com/doc — official reference for every command with clear examples

GitHub Guides

docs.github.com — pull requests, Actions, and team workflow tutorials

Copilot Docs

docs.github.com/copilot — setup guide and tips for maximizing suggestions

Conventional Commits

conventionalcommits.org — the specification for structured commit messages



See You on Day 3

You've built the version control foundation. Tomorrow, you'll automate everything — turning every git push into a fully automated build, test, and deploy pipeline with Jenkins on AWS.



DAY 3: JENKINS CI/CD PIPELINES